

Top 100 Java Coding Questions

Service-Based Company–Centric Practice Set • Coding Questions Only • No Solutions

Focus: Common fresher/entry-level coding patterns used in service-based company assessments and interviews. Difficulty tags are practice-oriented, not official company classifications.

Instruction: Solve every problem in Java. No Java theory, OOP definitions, or solutions are included.

1. Arrays

1. **[Easy]** Find the largest and smallest element in an integer array.
2. **[Easy]** Find the second largest element in an array without sorting the array.
3. **[Easy]** Check whether an array is sorted in ascending order.
4. **[Easy]** Reverse an array in place.
5. **[Easy]** Calculate the sum and average of all elements in an array.
6. **[Easy]** Count the number of even and odd elements in an array.
7. **[Easy]** Find all duplicate elements in an integer array.
8. **[Medium]** Remove duplicate elements from an array while preserving their first occurrence order.
9. **[Medium]** Find the missing number from an array containing distinct numbers from 1 to N.
10. **[Medium]** Move all zeroes to the end of an array while maintaining the order of non-zero elements.
11. **[Medium]** Find the pair of elements whose sum is equal to a given target.
12. **[Medium]** Find the intersection of two integer arrays.
13. **[Medium]** Find the union of two integer arrays without duplicate values.

2. Strings

14. **[Easy]** Reverse a string without using a library reverse method.
15. **[Easy]** Check whether a string is a palindrome.
16. **[Easy]** Count vowels, consonants, digits, spaces, and special characters in a string.
17. **[Easy]** Count the frequency of each character in a string.
18. **[Easy]** Find the first non-repeating character in a string.
19. **[Easy]** Remove duplicate characters from a string while preserving order.
20. **[Easy]** Check whether two strings are anagrams.
21. **[Medium]** Find the longest word in a sentence.
22. **[Medium]** Reverse the words of a sentence while preserving each word's characters.
23. **[Medium]** Find the most frequent character in a string.
24. **[Medium]** Check whether one string is a rotation of another.
25. **[Medium]** Find the longest substring without repeating characters.
26. **[Medium]** Count the number of occurrences of a given substring in a string.

3. Searching & Sorting

27. **[Easy]** Implement linear search and return the index of a target element.
28. **[Easy]** Implement binary search on a sorted array.
29. **[Easy]** Find the first and last occurrence of a target in a sorted array.

- 30. **[Easy]** Sort an array using bubble sort.
- 31. **[Easy]** Sort an array using selection sort.
- 32. **[Easy]** Sort an array using insertion sort.
- 33. **[Medium]** Merge two sorted arrays into one sorted array.
- 34. **[Medium]** Merge overlapping intervals.
- 35. **[Medium]** Find the Kth smallest element in an array.
- 36. **[Medium]** Sort an array containing only 0s, 1s, and 2s.

4. Hashing / Frequency

- 37. **[Easy]** Find the frequency of every integer in an array.
- 38. **[Easy]** Find the element that occurs only once when every other element occurs twice.
- 39. **[Easy]** Find all pairs of elements with a given sum using a frequency map.
- 40. **[Medium]** Find the longest subarray with sum equal to K.
- 41. **[Medium]** Count the number of subarrays whose sum equals K.
- 42. **[Medium]** Find the first repeated element in an array.
- 43. **[Medium]** Group a list of strings into anagram groups.

5. Two Pointers & Sliding Window

- 44. **[Easy]** Check whether a sorted array contains a pair with a given sum using two pointers.
- 45. **[Easy]** Remove duplicates from a sorted array in place.
- 46. **[Medium]** Find the maximum sum of any contiguous subarray of size K.
- 47. **[Medium]** Find the minimum length subarray whose sum is at least a target.
- 48. **[Medium]** Find the longest substring containing at most K distinct characters.
- 49. **[Medium]** Find the longest subarray containing equal numbers of 0s and 1s.
- 50. **[Medium]** Find all starting indices of anagrams of a pattern in a string.

6. Matrix / 2D Arrays

- 51. **[Easy]** Print all elements of a matrix row by row.
- 52. **[Easy]** Find the sum of the primary and secondary diagonals.
- 53. **[Easy]** Transpose a square matrix.
- 54. **[Easy]** Search for an element in a row-wise and column-wise sorted matrix.
- 55. **[Medium]** Rotate a square matrix by 90 degrees clockwise.
- 56. **[Medium]** Print a matrix in spiral order.
- 57. **[Medium]** Print a matrix in boundary order.

7. Linked List

- 58. **[Easy]** Create a singly linked list and print all its elements.
- 59. **[Easy]** Find the length of a singly linked list.
- 60. **[Easy]** Search for a value in a linked list.
- 61. **[Easy]** Reverse a singly linked list.
- 62. **[Medium]** Find the middle node of a linked list.
- 63. **[Medium]** Detect whether a linked list contains a cycle.

64. [Medium] Remove the Nth node from the end of a linked list.

8. Stack & Queue

65. [Easy] Implement a stack using an array.

66. [Easy] Implement a queue using an array.

67. [Easy] Check whether brackets in an expression are balanced.

68. [Easy] Reverse a string using a stack.

69. [Medium] Implement a stack using two queues.

70. [Medium] Implement a queue using two stacks.

71. [Medium] Find the next greater element for every element in an array.

9. Recursion & Basic Backtracking

72. [Easy] Calculate factorial of N using recursion.

73. [Easy] Print the first N Fibonacci numbers using recursion.

74. [Easy] Find the sum of digits of a number using recursion.

75. [Easy] Reverse an integer using recursion.

76. [Medium] Generate all subsequences of a string.

77. [Medium] Generate all permutations of a string.

78. [Medium] Generate all valid combinations of N pairs of parentheses.

10. Numbers / Mathematics

79. [Easy] Check whether a number is prime.

80. [Easy] Print all prime numbers in a given range.

81. [Easy] Check whether a number is an Armstrong number.

82. [Easy] Check whether a number is a palindrome.

83. [Easy] Find GCD and LCM of two numbers.

84. [Easy] Reverse the digits of an integer.

85. [Medium] Check whether a number is a perfect number.

11. Trees / BST

86. [Easy] Perform preorder, inorder, and postorder traversal of a binary tree.

87. [Easy] Find the height of a binary tree.

88. [Medium] Perform level-order traversal of a binary tree.

89. [Medium] Count leaf nodes in a binary tree.

90. [Medium] Check whether two binary trees are identical.

91. [Medium] Search for a value in a binary search tree.

92. [Medium] Find the minimum and maximum value in a binary search tree.

12. Dynamic Programming / Common Interview Patterns

93. [Medium] Find the number of ways to climb N stairs when you can take 1 or 2 steps.

94. [Medium] Find the maximum sum of non-adjacent elements in an array.

95. [Medium] Solve the 0/1 knapsack problem.

96. **[Medium]** Find the length of the longest common subsequence of two strings.
97. **[Medium]** Find the length of the longest increasing subsequence of an array.
98. **[Hard]** Find the minimum number of coins required to make a given amount.
99. **[Hard]** Determine whether an array can be partitioned into two subsets with equal sum.
100. **[Hard]** Find the minimum edit distance between two strings.

Coverage: Arrays • Strings • Searching & Sorting • Hashing • Two Pointers & Sliding Window • Matrices • Linked Lists • Stack & Queue • Recursion & Backtracking • Mathematics • Trees/BST • Dynamic Programming